

Question 11

Q: Compare two image acquisition systems (CCD vs CMOS) in terms of their impact on image quality in practical applications.

Theoretical Concept: CCD vs CMOS Sensors

CCD (Charge-Coupled Device): Traditionally offers higher image quality, less noise, and better light sensitivity because every pixel's charge is processed by a single high-quality amplifier. (Used in medical/astro imaging).

CMOS (Complementary Metal-Oxide Semiconductor): Processes charge at the individual pixel level. Faster, cheaper, and consumes less power. Modern CMOS sensors rival CCDs in quality and are used in almost all smartphones and digital cameras today.

OpenCV Python Solution

 Copy

```
import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt # Import Matplotlib to visualize the output on screen

# Step 1: Create a clean base image
img = np.full((100, 100), 100, dtype=np.uint8) # Create a solid gray image array filled with a value

# Step 2: Simulate a high-end CCD sensor (Virtually zero read noise)
```

```
ccd_img = img.copy()

# Step 3: Simulate an older/cheap CMOS sensor (High read noise and thermal noise)
noise = np.random.normal(0, 15, img.shape).astype(np.int16) # Generate random numbers to simulate sensor noise
cmos_img = np.clip(img.astype(np.int16) + noise, 0, 255).astype(np.uint8) # Clamp all values into a valid pixel range

# Step 4: Compare the acquisitions side by side
plt.figure(figsize=(8,4)) # Create a new figure window for display
plt.subplot(121), plt.imshow(ccd_img, cmap='gray'), plt.title('Simulated CCD (Clean)') # Place this image in a subplot
plt.subplot(122), plt.imshow(cmos_img, cmap='gray'), plt.title('Simulated CMOS (Noisy)') # Place this image in a subplot
plt.show() # Show the final plot window to the user
```

Expected Output

A visual simulation of hardware sensor output: the CCD image will be a perfectly smooth flat gray, while the CMOS image will be visually speckled due to individual pixel read-noise interference.

How to Execute & Submit

1. Google Colab Execution:

- Open [Google Colab](#) and create a New Notebook.
- Click the  **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

2. Uploading to GitHub:

- In Colab, go to **File > Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.